

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	G.Srivardhan	Reg. No.:	192425208
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Aim:

To implement and compare two different greedy algorithmic strategies: Job Sequencing (to maximize profit under deadline constraints) and Container Loading (to maximize the number of items loaded within a fixed capacity). The study includes an analysis of selection strategies, optimality, and computational complexity.

Procedure:

1. Job Sequencing Implementation:
 - Represent jobs as a list of tuples containing (profit, deadline).

- Sort the jobs in descending order based on profit.
 - Initialize a time-slot array based on the maximum deadline found in the input.
 - For each job, iterate backwards from its deadline to find the first available free slot.
 - If a slot is found, assign the job to that slot and add its profit to the total.
2. Container Loading Implementation:
- Represent container weights as a list of integers.
 - Sort the weights in ascending order (the greedy choice to maximize count).
 - Iterate through the sorted weights, adding them to the container one by one.
 - Stop adding weights if the next weight would exceed the remaining capacity.
3. Comparative Analysis:
- Compare the greedy criteria used (Profit-based vs. Weight-based).
 - Evaluate the optimality of each approach relative to its specific objective.
 - Analyze the Time Complexity of both algorithms.

Code:

```
# ----- Job Sequencing Problem (Greedy by Profit) -----
def job_sequencing(jobs):
    jobs = sorted(jobs, key=lambda x: x[0], reverse=True)
    max_deadline = max(job[1] for job in jobs)
    slots = [None] * (max_deadline + 1)
    total_profit = 0

    for profit, deadline in jobs:
        for j in range(deadline, 0, -1):
            if slots[j] is None:
                slots[j] = (profit, deadline)
                total_profit += profit
                break

    selected_jobs = [job for job in slots if job is not None]
    return selected_jobs, total_profit

# ----- Container Loading Problem (Greedy by Weight) -----
def container_loading(weights, capacity):
    weights = sorted(weights)
    loaded = []
    total_weight = 0

    for w in weights:
        if total_weight + w <= capacity:
            loaded.append(w)
            total_weight += w
        else:
            break

    return loaded, total_weight

# ----- Main Execution -----
if __name__ == "__main__":
```

```
jobs = [(100, 2), (19, 1), (27, 2), (25, 1), (15, 3)]
weights = [10, 20, 30, 40]
capacity = 60
```

```
selected_jobs, total_profit = job_sequencing(jobs)
loaded_containers, total_loaded_weight = container_loading(weights, capacity)
```

```
print("=== Job Sequencing Problem ===")
print(f'Selected Jobs (profit, deadline): {selected_jobs}')
print(f'Total Profit: {total_profit}')
```

```
print("\n=== Container Loading Problem ===")
print(f'Loaded Containers: {loaded_containers}')
print(f'Total Weight Loaded: {total_loaded_weight}')
print(f'Remaining Capacity: {capacity - total_loaded_weight}')
```

```
print("\n=== Comparative Analysis ===")
print("1. Strategies: Job Sequencing uses High-Profit greedy; Container Loading uses Low-Weight greedy.")
print("2. Optimality: Job Sequencing is optimal for profit; Container Loading is optimal for item count.")
print("3. Complexity: Job Sequencing is  $O(n^2)$ ; Container Loading is  $O(n \log n)$ .")
Result:
```

The algorithms were executed successfully with the following outputs:

```
=== Job Sequencing Problem ===
Selected Jobs (profit, deadline): [(27, 2), (100, 2), (15, 3)]
Total Profit: 142
```

```
=== Container Loading Problem ===
Loaded Containers: [10, 20, 30]
Total Weight Loaded: 60
Remaining Capacity: 0
```

ASSESSMENT CO4-AT3-COMPARATIVE ANALYSIS

Back

Language

Python

C

C++

Java

QUESTIONS

Q1

Job Sequencing vs Container Loading (Greedy Comparison)

100 marks

1/1 answered

Q1 Job Sequencing vs Container Loading (Greedy Comparison)

100 marks

Given jobs (profit, deadline) [(100,2),(19,1),(27,2),(25,1),(15,3)], solve Job Sequencing. Also load containers with weights [10,20,30,40] into capacity 50 using greedy. Compare selection strategies and outputs. Analyze optimality. Evaluate complexity.

Your Code

```

def job_sequencing(jobs):
    jobs = sorted(jobs, key=lambda x: x[0], reverse=True)
    max_deadline = max(job[1] for job in jobs)
    slot = [False] * (max_deadline + 1)
    job_sequence = [None] * (max_deadline + 1)
    total_profit = 0

    for profit, deadline in jobs:
        # Find a free slot from deadline down to 1
        for j in range(deadline, 0, -1):
            if not slot[j]:
                slot[j] = True
                job_sequence[j] = (profit, deadline)
                total_profit += profit
                break

    selected_jobs = [job for job in job_sequence if job is not None]
    return selected_jobs, total_profit

```

Input

stdin input

Output

```

=== Job Sequencing Problem ===
Selected Jobs (profit, deadline):
[(27, 2), (100, 1), (15, 3)]
Total Profit: 142

```

=== Comparative Analysis ===

1. Strategies: Job Sequencing uses High-Profit greedy; Container Loading uses Low-Weight greedy.
2. Optimality: Job Sequencing is optimal for profit; Container Loading is optimal for item count.
3. Complexity: Job Sequencing is $O(n^2)$; Container Loading is $O(n \log n)$.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____

Date: _____

Date: _____

Date: _____